

TUAR_Artifacts_Corpus

August 7, 2026

1 Artifacts analysis

```
[1]: %matplotlib inline

# Data integration libraries
from pathlib import Path

# Data analysis libraries
import mne
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# Data visualization and control libraries
from IPython.display import display
import time
```

1.0.1 Functions from general analysis

```
[2]: # Path of the TUH EEG Corpus
#base_path = Path(r"F:\MAFER\UVG\TRABAJO DE GRADUACIÓN MECATRÓNICA\TUH_EEG")
base_path = Path(r"D:\1\tuh")

# TUH Corpus structure
CORPUS_PATHS = {
    "all_corpus": Path("tuh_eeg")/"v2.0.2",
    "artifact": Path("tuh_eeg_artifact")/"v3.0.1",
    "epilepsy": Path("tuh_eeg_epilepsy")/"v3.1.0",
    "seizure": Path("tuh_eeg_seizure")/"v2.0.6",
    "events": Path("tuh_eeg_events")/"v2.0.1",
}

# Pandas table display options
pd.set_option("display.max_columns", None)
pd.set_option("display.max_colwidth", 20)
pd.set_option("display.width", 1000)
```

1.1 TUH EEG Artifact Corpus EDA

This subset contains annotations of 5 different artifacts: - Eye movement - Chewing - Shivering - Electrode pop, electrode static and lead artifacts - Muscle artifacts

1.1.1 EDF files

```
[3]: # Corpus signal data
def get_edf_data(base_path, corpus_name, n_seconds=10):
    """
    Returns all signal parameters of a specific corpus.
    """
    corpus_path = base_path / CORPUS_PATHS[corpus_name]
    montages = [
        "01_tcp_ar", "02_tcp_le", "03_tcp_ar_a", "04_tcp_le_a",
        "05_tcp_ar_b", "06_tcp_le_b", "07_tcp_ar_c", "08_tcp_le_c"
    ]

    registers = []

    for session in corpus_path.rglob("*.edf"):
        montage = session.parent.name if session.parent.name in montages else None
        document_name = Path(session.name).stem
        session_id = next((part for part in document_name.split("_") if part.
        startswith("s") and part[1:].isdigit()), None)
        patient = next((part for part in document_name.split("_") if len(part)
        == 8 and part.isalpha()), None)
        session_section = next((part for part in document_name.split("_") if
        part.startswith("t") and part[1:].isdigit()), None)

        # Load EDF file data (Preload = False)
        mne.set_log_level("WARNING")
        edf_data = mne.io.read_raw_edf(session, preload=False,
        verbose="WARNING")

        registers.append({
            "Corpus": corpus_name,
            "Montage": montage,
            "Session": session_id,
            "Patient": patient,
            "Section": session_section,
            "Channels": edf_data.ch_names,
            "Amount of channels": len(edf_data.ch_names),
            "Sampling frequency [Hz]": edf_data.info["sfreq"],
            "Duration [s]": edf_data.times[-1] if len(edf_data.times) > 0 else
            np.nan,
            "Total amount of sampling data": edf_data.n_times,
```

```

    })

    return pd.DataFrame(registers)

# EDFs Data extraction
signal_sessions_artifact_df = get_edf_data(base_path, "artifact")

```

```

[4]: # Visualization of previews extracted data
print(">> Sessions database preview")
display(signal_sessions_artifact_df.head(5))

```

>> Sessions database preview

	Corpus	Montage	Session	Patient	Section	Channels	Amount of
	channels	Sampling frequency	[Hz]	Duration [s]	Total amount of sampling data		
0	artifact	01_tcp_ar	s005	aaaaaaju	t000	[EEG FP1-REF, EE...	
	↪ 36		250.0	1441.996000	360500		
1	artifact	01_tcp_ar	s014	aaaaadmi	t008	[EEG FP1-REF, EE...	
	↪ 29		256.0	1260.996094	322816		
2	artifact	01_tcp_ar	s007	aaaaaaju	t000	[EEG FP1-REF, EE...	
	↪ 30		250.0	1243.996000	311000		
3	artifact	01_tcp_ar	s004	aaaaaall	t000	[EEG FP1-REF, EE...	
	↪ 30		250.0	1449.996000	362500		
4	artifact	01_tcp_ar	s002	aaaaaayg	t001	[EEG FP1-REF, EE...	
	↪ 32		400.0	549.997500	220000		

EDF Characteristics

```

[14]: %matplotlib inline
## EDA Visualization
# General function to plot visualization for the given characteristic of the
↪ EDF Artifact corpus analysis
def plot_visualization(column_name, analyzed_parameter):
    """Plot visualization for the given dataframe column"""
    data = signal_sessions_artifact_df[column_name].value_counts().sort_index()
    axes = pd.DataFrame(data).reset_index().plot.bar(
        x=column_name,
        rot=0,
        title=f"EDF {analyzed_parameter} distribution",
        xlabel=f"EDF {analyzed_parameter} values",
        ylabel="Amount of sessions according to the EDF files",
        legend=False,
    )

    for container in axes.containers:
        axes.bar_label(container, label_type="edge", fontsize=8, padding=3)

    return axes

```

```

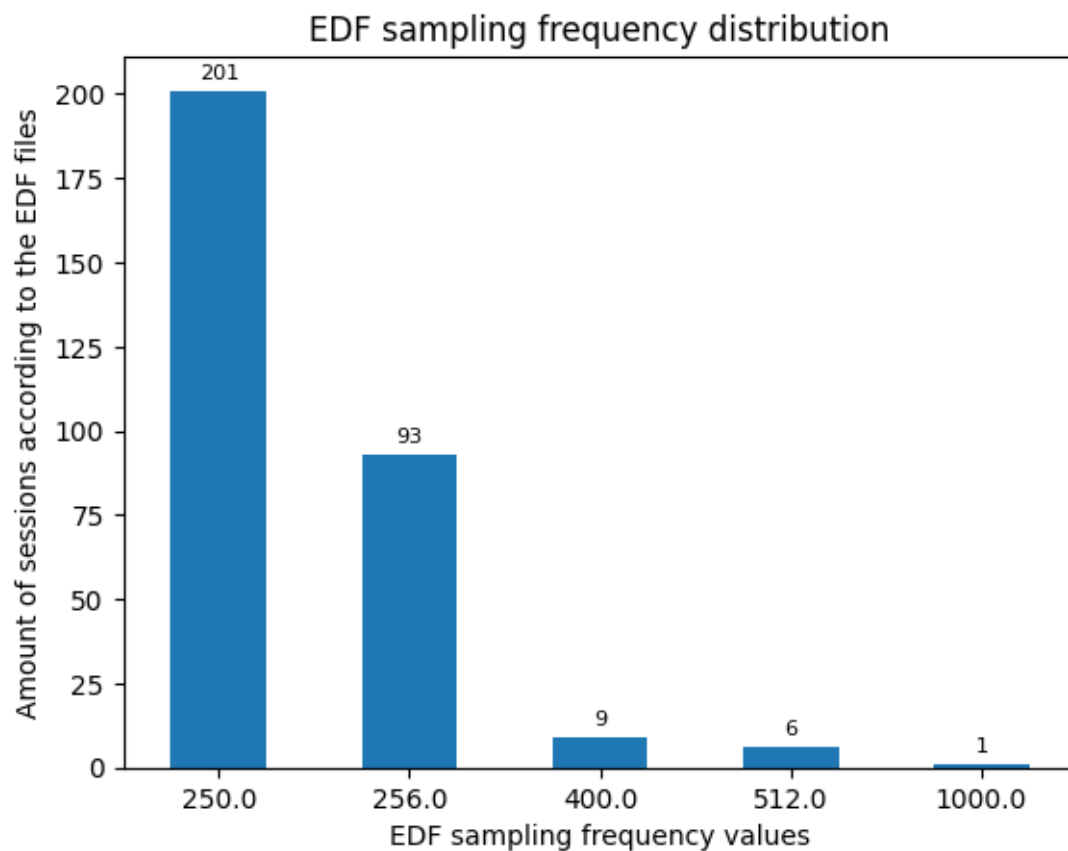
# Frequency distribution of the sampling frequency
print("\n>> Distribution of the sampling frequency")
plot_visualization("Sampling frequency [Hz]", "sampling frequency")
plt.show()

# Channels distribution - examination of correct data channels
print("\n>> Distribution of the amount of channels")
plot_visualization("Amount of channels", "amount of channels")
plt.show()

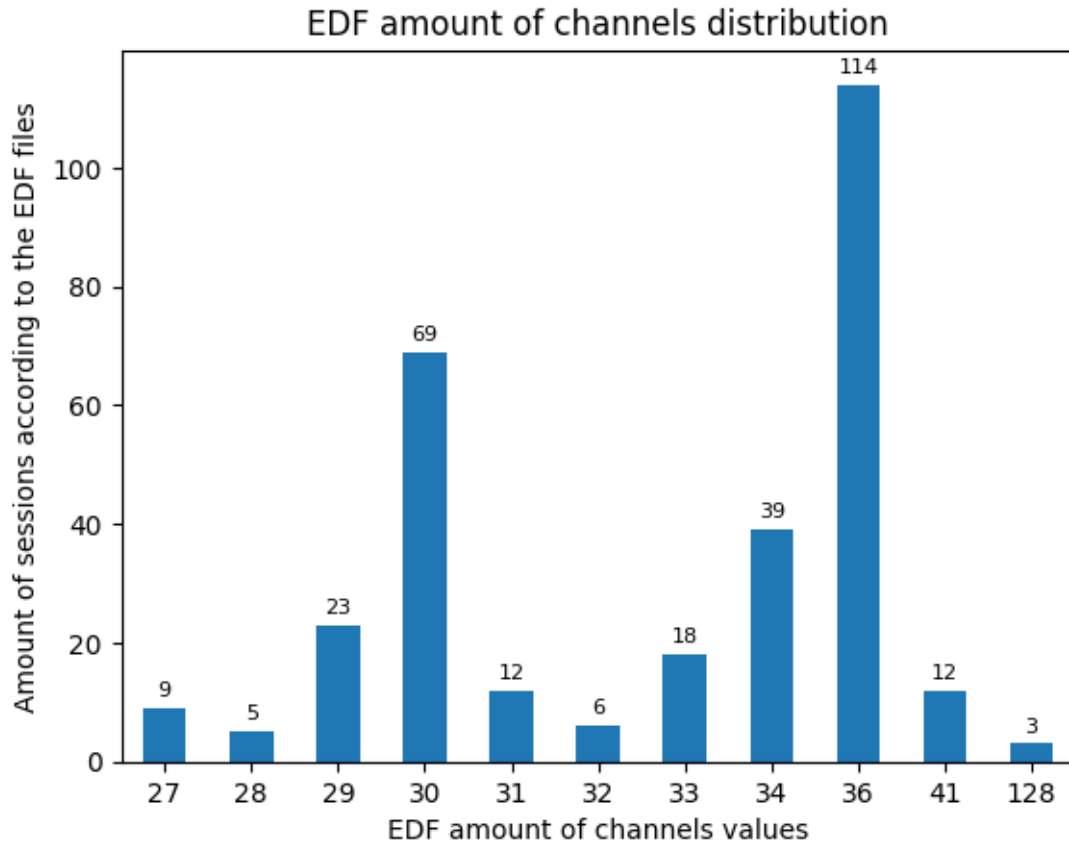
# Examination of representative channels accordint to the amount found by the
↳ distribution
exploded = signal_sessions_artifact_df[["Amount of channels", "Channels"]].
↳ explode("Channels").sort_values(by="Amount of channels").
↳ reset_index(drop=True)
channel_matrix = pd.crosstab(exploded["Channels"], exploded["Amount of
↳ channels"])
channel_matrix["Total files"] = channel_matrix.drop(columns=[c for c in
↳ channel_matrix.columns if c == "Total files"], errors="ignore").sum(axis=1)
channel_matrix["N variants present"] = (channel_matrix.drop(columns="Total
↳ files") > 0).sum(axis=1)
channel_matrix = channel_matrix.sort_values(["N variants present", "Total
↳ files"], ascending=False)
print("\n>> Channel matrix preview")
display(channel_matrix.head(5))

```

>> Distribution of the sampling frequency



>> Distribution of the amount of channels



>> Channel matrix preview

Amount of channels	27	28	29	30	31	32	33	34	36	41	128	Total files	N
variants present													
Channels													
EEG C3-REF	9	5	23	69	12	6	17	39	114	0	3	297	
↪	10												
EEG C4-REF	9	5	23	69	12	6	17	39	114	0	3	297	
↪	10												
EEG CZ-REF	9	5	23	69	12	6	17	39	114	0	3	297	
↪	10												
EEG F3-REF	9	5	23	69	12	6	17	39	114	0	3	297	
↪	10												
EEG F4-REF	9	5	23	69	12	6	17	39	114	0	3	297	
↪	10												

Preview of EDF content

[16]: %matplotlib qt

```

# Manual analysis of sessions with 128 channels to determine the most
↳ representative channels for the corpus
print("\n>> Sessions with 128 channels")
display(signal_sessions_artifact_df.loc[signal_sessions_artifact_df["Amount of
↳ channels"] == 27])

print("\n>> Plotting a sample EDF file from the Artifact corpus with 128
↳ channels")
raw = mne.io.read_raw_edf(
    base_path / CORPUS_PATHS["artifact"] / "edf" / "03_tcp_ar_a" /
↳ "aaaaabnn_s002_t001.edf",
    preload=False,
    verbose="WARNING",
)
print(raw.info)
print("Channels:", raw.ch_names)
print("Channel count:", len(raw.ch_names))

raw.plot()

```

>> Sessions with 128 channels

	Corpus	Montage	Session	Patient	Section	Channels	Amount
	of channels	Sampling frequency	[Hz]	Duration [s]	Total amount of sampling		
	↳ data						
8	artifact	01_tcp_ar	s006	aaaaael	t006	[EEG FP1-REF, EE...	
↳ 27		256.0	659.996094	168960			
35	artifact	01_tcp_ar	s007	aaaaaezj	t002	[EEG FP1-REF, EE...	
↳ 27		512.0	600.998047	307712			
157	artifact	01_tcp_ar	s010	aaaaamhb	t005	[EEG FP1-REF, EE...	
↳ 27		256.0	600.996094	153856			
193	artifact	01_tcp_ar	s002	aaaaaoav	t000	[EEG FP1-REF, EE...	
↳ 27		256.0	1500.996094	384256			
219	artifact	01_tcp_ar	s001	aaaaanyf	t001	[EEG FP1-REF, EE...	
↳ 27		256.0	1367.996094	350208			
224	artifact	01_tcp_ar	s002	aaaaaora	t000	[EEG FP1-REF, EE...	
↳ 27		256.0	1499.996094	384000			
243	artifact	01_tcp_ar	s002	aaaaapks	t003	[EEG FP1-REF, EE...	
↳ 27		256.0	600.996094	153856			
251	artifact	01_tcp_ar	s001	aaaaaovl	t000	[EEG FP1-REF, EE...	
↳ 27		256.0	1356.996094	347392			
284	artifact	01_tcp_ar	s001	aaaaapre	t002	[EEG FP1-REF, EE...	
↳ 27		256.0	1241.996094	317952			

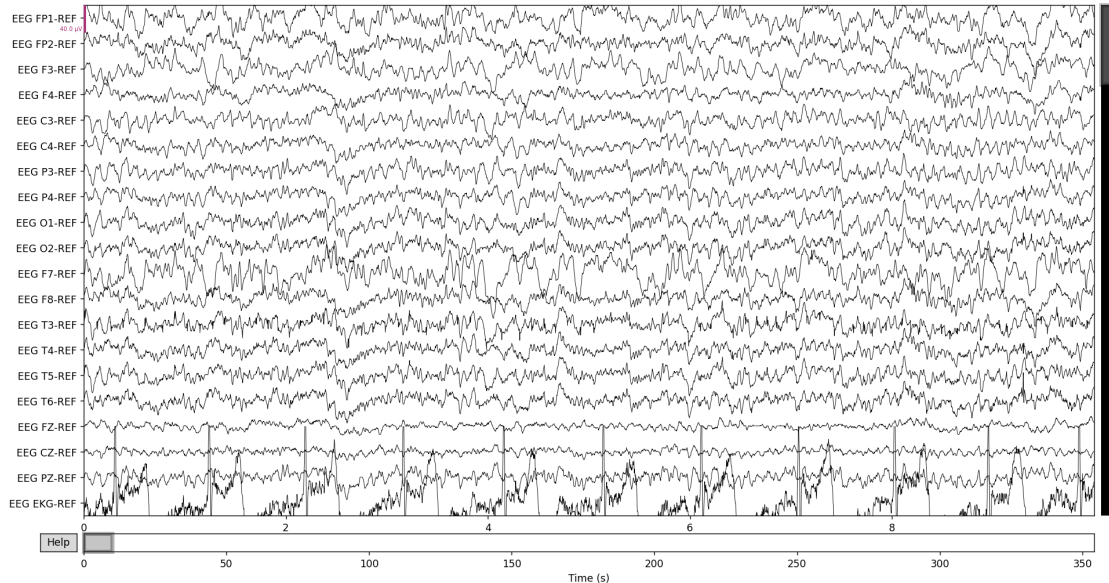
>> Plotting a sample EDF file from the Artifact corpus with 128 channels
<Info | 8 non-empty values

```

bads: []
ch_names: EEG FP1-REF, EEG FP2-REF, EEG F3-REF, EEG F4-REF, EEG C3-REF, ...
chs: 128 EEG
custom_ref_applied: False
highpass: 0.0 Hz
lowpass: 200.0 Hz
meas_date: 2000-01-01 00:00:00 UTC
nchan: 128
projs: []
sfreq: 400.0 Hz
subject_info: <subject_info | his_id: aaaaabnn, sex: 1, last_name: aaaaabnn>
>
Channels: ['EEG FP1-REF', 'EEG FP2-REF', 'EEG F3-REF', 'EEG F4-REF', 'EEG
C3-REF', 'EEG C4-REF', 'EEG P3-REF', 'EEG P4-REF', 'EEG O1-REF', 'EEG O2-REF',
'EEG F7-REF', 'EEG F8-REF', 'EEG T3-REF', 'EEG T4-REF', 'EEG T5-REF', 'EEG
T6-REF', 'EEG FZ-REF', 'EEG CZ-REF', 'EEG PZ-REF', 'EEG EKG-REF', 'EEG 21-REF',
'EEG 22-REF', 'EEG T1-REF', 'EEG T2-REF', 'EEG 25-REF', 'EEG 26-REF', 'EEG
27-REF', 'EEG 28-REF', 'EEG 29-REF', 'EEG 30-REF', 'EEG 31-REF', 'EEG 32-REF',
'EEG 33-REF', 'EEG 34-REF', 'EEG 35-REF', 'EEG 36-REF', 'EEG 37-REF', 'EEG
38-REF', 'EEG 39-REF', 'EEG 40-REF', 'EEG 41-REF', 'EEG 42-REF', 'EEG 43-REF',
'EEG 44-REF', 'EEG 45-REF', 'EEG 46-REF', 'EEG 47-REF', 'EEG 48-REF', 'EEG
49-REF', 'EEG 50-REF', 'EEG 51-REF', 'EEG 52-REF', 'EEG 53-REF', 'EEG 54-REF',
'EEG 55-REF', 'EEG 56-REF', 'EEG 57-REF', 'EEG 58-REF', 'EEG 59-REF', 'EEG
60-REF', 'EEG 61-REF', 'EEG 62-REF', 'EEG 63-REF', 'EEG 64-REF', 'EEG 65-REF',
'EEG 66-REF', 'EEG 67-REF', 'EEG 68-REF', 'EEG 69-REF', 'EEG 70-REF', 'EEG
71-REF', 'EEG 72-REF', 'EEG 73-REF', 'EEG 74-REF', 'EEG 75-REF', 'EEG 76-REF',
'EEG 77-REF', 'EEG 78-REF', 'EEG 79-REF', 'EEG 80-REF', 'EEG 81-REF', 'EEG
82-REF', 'EEG 83-REF', 'EEG 84-REF', 'EEG 85-REF', 'EEG 86-REF', 'EEG 87-REF',
'EEG 88-REF', 'EEG 89-REF', 'EEG 90-REF', 'EEG 91-REF', 'EEG 92-REF', 'EEG
93-REF', 'EEG 94-REF', 'EEG 95-REF', 'EEG 96-REF', 'EEG 97-REF', 'EEG 98-REF',
'EEG 99-REF', 'EEG 100-REF', 'EEG 101-REF', 'EEG 102-REF', 'EEG 103-REF', 'EEG
104-REF', 'EEG 105-REF', 'EEG 106-REF', 'EEG 107-REF', 'EEG 108-REF', 'EEG
109-REF', 'EEG 110-REF', 'EEG 111-REF', 'EEG 112-REF', 'EEG 113-REF', 'EEG
114-REF', 'EEG 115-REF', 'EEG 116-REF', 'EEG 117-REF', 'EEG 118-REF', 'EEG
119-REF', 'EEG 120-REF', 'EEG 121-REF', 'EEG 122-REF', 'EEG 123-REF', 'EEG
124-REF', 'EEG 125-REF', 'EEG 126-REF', 'EEG 127-REF', 'EEG 128-REF']
Channel count: 128

```

[16]:



Patients distribution and characteristics

```
[ ]: # Sessions reported per patient
print("\n>> Sessions per patient")
sessions_per_patient = signal_sessions_artifact_df["Patient"].value_counts()
display(sessions_per_patient.describe())

# Amount of sessions per patient
print("\n>> Sessions per patient database (head)")
display(signal_sessions_artifact_df["Patient"].value_counts().head(30))
```

>> Sessions per patient

```
count    213.000000
mean      1.455399
std       1.183268
min       1.000000
25%       1.000000
50%       1.000000
75%       1.000000
max       10.000000
Name: count, dtype: float64
```

>> Sessions per patient database (head)

```
Patient
aaaaaanrc    10
aaaaajrj     7
```

```

aaaaamhb      7
aaaaapas      6
aaaaabbn      5
aaaaajns      5
aaaaadmi      4
aaaaabiw      4
aaaaajrf      4
aaaaajqo      4
aaaaamcr      4
aaaaamyc      4
aaaaappv      4
aaaaacyf      4
aaaaafcf      3
aaaaaimu      3
aaaaakbz      3
aaaaakeq      3
aaaaalzg      3
aaaaanhb      3
aaaaaogd      3
aaaaapks      3
aaaaagqb      3
aaaaabnn      3
aaaaaaju      2
aaaaagsc      2
aaaaahte      2
aaaaajah      2
aaaaajqu      2
aaaaalbt      2
Name: count, dtype: int64

```

1.1.2 Annotations files

```

[32]: ## Annotations
      # Format analysis of the annotations
      path = base_path / CORPUS_PATHS["artifact"] / "edf" / "03_tcp_ar_a" /
        ↪ "aaaaabnn_s002_t001.csv"

      text = path.read_text(encoding="utf-8", errors="replace")
      comments = [line for line in text.splitlines() if line.startswith("#")]
      print(comments)

      ann = pd.read_csv(
          path,
          sep=",",
          comment="#"
      )

```

```
display(ann.head(5))
```

```
['# version = csv_v1.0.0', '# bname = aaaaabnn_s002_t001', '# duration =  
354.0000 secs', '# montage_file = $TUAR/v3.0.0/DOCS/03_tcp_ar_a_montage.txt', '#  
annotation_label_file = $TUAR/v3.0.0/DOCS/nedc_ann_eeg_tools_map_v01.txt', '#']
```

	channel	start_time	stop_time	label	confidence
0	FP1-F7	33.5759	150.2054	musc	1.0
1	FP1-F7	150.2656	151.9846	eyem	1.0
2	FP1-F7	152.2879	175.1674	musc	1.0
3	FP1-F7	176.7031	196.1228	musc	1.0
4	FP1-F7	252.8906	278.0022	musc	1.0

```
[33]: # Corpus annotations data  
def get_csv_data(base_path, corpus_name):  
    """  
    Returns all signal annotation marks of a specific corpus.  
    """  
    corpus_path = base_path / CORPUS_PATHS[corpus_name]  
    montages = [  
        "01_tcp_ar", "02_tcp_le", "03_tcp_ar_a", "04_tcp_le_a",  
        "05_tcp_ar_b", "06_tcp_le_b", "07_tcp_ar_c", "08_tcp_le_c"  
    ]  
  
    registers = []  
  
    for session in corpus_path.rglob("*.csv"):  
        ann = pd.read_csv(session, sep=",", comment="#")  
  
        text = session.read_text(encoding="utf-8", errors="replace")  
        comments = [line for line in text.splitlines() if line.startswith("#")]  
  
        montage = session.parent.name if session.parent.name in montages else_  
        ↪None  
        document_name = Path(session.name).stem  
        session_id = next((part for part in document_name.split("_") if part.  
        ↪startswith("s") and part[1:].isdigit()), None)  
        patient = next((part for part in document_name.split("_") if len(part)_  
        ↪== 8 and part.isalpha()), None)  
        session_section = next((part for part in document_name.split("_") if_  
        ↪part.startswith("t") and part[1:].isdigit()), None)  
  
        ann = ann.assign(  
            Montage=montage,  
            Session=session_id,  
            Patient=patient,  
            Section=session_section,  
            WindowSize= lambda x: x["stop_time"] - x["start_time"],
```

```

    )

    registers.append(ann)

    return pd.concat(registers, ignore_index=True)

# CSVs Data extraction
data_sessions_artifact_df = get_csv_data(base_path, "artifact")
print("\n>> Sessions database preview")
display(data_sessions_artifact_df.head(5))

```

>> Sessions database preview

	channel	start_time	stop_time	label	confidence	Montage	Session
	↳Patient	Section	WindowSize				
0	FP1-F7	440.4222	454.0444	musc	1.0	01_tcp_ar	s014
	↳aaaaadmi	t008	13.6222				
1	FP1-F7	454.2222	457.3333	musc_elec	1.0	01_tcp_ar	s014
	↳aaaaadmi	t008	3.1111				
2	FP1-F7	457.6444	477.4333	musc	1.0	01_tcp_ar	s014
	↳aaaaadmi	t008	19.7889				
3	FP1-F7	482.4889	503.3222	musc	1.0	01_tcp_ar	s014
	↳aaaaadmi	t008	20.8333				
4	FP1-F7	503.3667	505.9000	musc_elec	1.0	01_tcp_ar	s014
	↳aaaaadmi	t008	2.5333				

```

[53]: # All registered labels in the Artifact corpus
display(pd.DataFrame({"label": data_sessions_artifact_df["label"].unique()}))

```

	label
0	musc
1	musc_elec
2	eyem_musc
3	elec
4	eyem
5	bckg
6	tcsz
7	fnsz
8	elpp
9	eyem_elec
10	chew
11	eyem_shiv
12	shiv
13	shiv_elec
14	eyem_chew
15	chew_musc
16	gnsz

```

17      cpsz
18  chew_elec

```

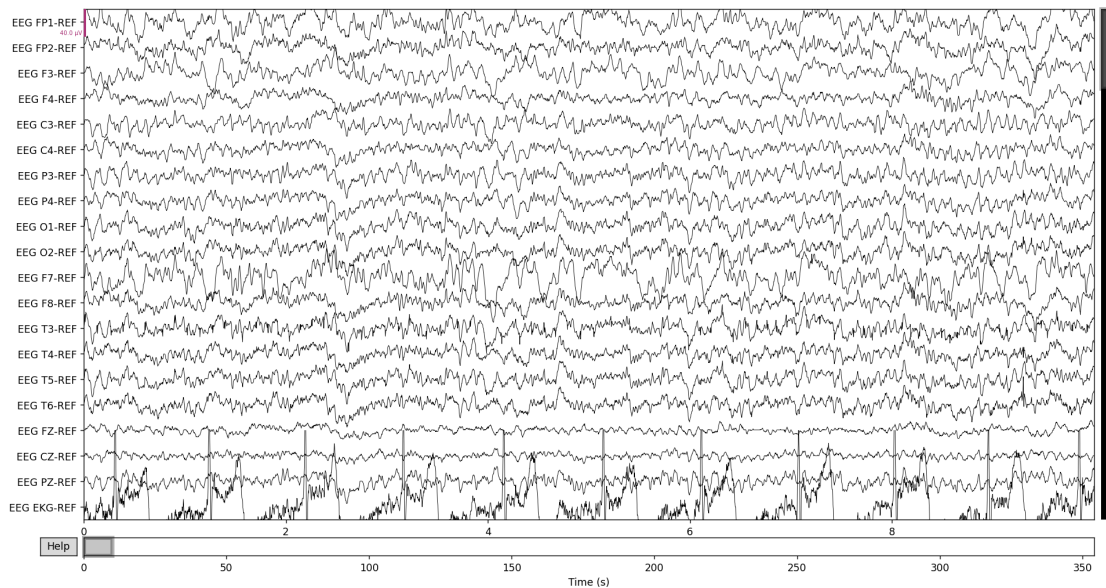
```

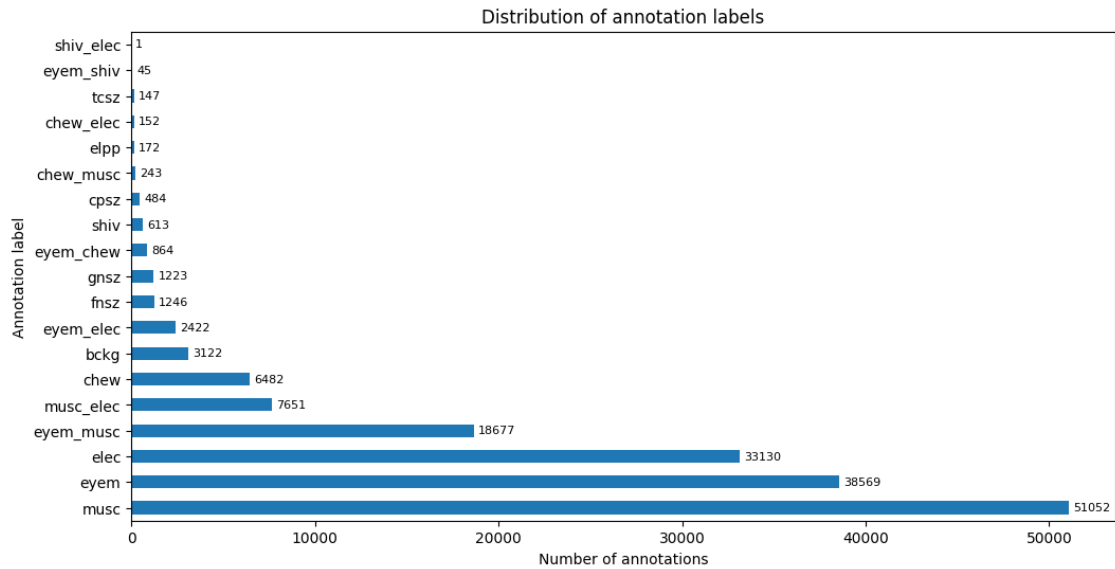
[10]: # Annotations distribution
      %matplotlib inline

data = data_sessions_artifact_df["label"].value_counts()
axes = pd.DataFrame(data).reset_index().plot.barh(
    x="label",
    rot=0,
    figsize=(12, 6),
    title=f"Distribution of annotation labels",
    xlabel=f"Number of annotations",
    ylabel="Annotation label",
    legend=False,
)

for container in axes.containers:
    axes.bar_label(container, label_type="edge", fontsize=8, padding=3)
plt.show()

```





Según la información de la tabla, el dataset está balanceado (desbalance clásico), favoreciendo el aprendizaje de los artefactos “musc”, “eyem” y “elec” si no se analiza la información.

```
[11]: # Data correlation, annotations per session
data_sessions_artifact_df["RecordID"] = (
    data_sessions_artifact_df["Patient"] + "_" +
    data_sessions_artifact_df["Session"] + "_" +
    data_sessions_artifact_df["Section"]
)
data_sessions_artifact_df["Tokens"] = (
    data_sessions_artifact_df["label"]
    .str.split("_")
)
data_sessions_artifact_df["NLabels"] = (
    data_sessions_artifact_df["Tokens"]
    .str.len()
)

print("\n>> Count of different labels by session")
print(data_sessions_artifact_df["NLabels"].value_counts())

SEIZURE_TOKENS = {"tcsz", "cpsz", "gnsz", "fnsz"}

data_sessions_artifact_df["HasSeizure"] = (
    data_sessions_artifact_df["Tokens"]
    .apply(lambda x: any(label in SEIZURE_TOKENS for label in x))
)
```

```

ARTIFACT_TOKENS = {
    "eyem", "musc", "elec", "elpp", "chew", "shiv"
}

data_sessions_artifact_df["HasArtifact"] = (
    data_sessions_artifact_df["Tokens"]
    .apply(lambda x: any(label in ARTIFACT_TOKENS for label in x))
)

record_summary = (
    data_sessions_artifact_df
    .groupby("RecordID")
    .agg({
        "HasArtifact": "max",
        "HasSeizure": "max"
    })
)

print("\n>> Count of records with artifacts and seizures by session")
print(record_summary.value_counts())

```

```

>> Count of different labels by session
NLabels
1    136240
2     30055
Name: count, dtype: int64

```

```

>> Count of records with artifacts and seizures by session
HasArtifact  HasSeizure
True         False      268
            True       42
Name: count, dtype: int64

```

Según los datos obtenidos se puede distribuir la información para tener suficientes datos para cada subclase según las características comunes, en el siguiente segmento se propone una distribución de las 19 clases disponibles teniendo en cuenta la cantidad de información de cada segmento.

Annotations categorization

```

[54]: # Artifacts labels
ARTIFACT_KEYWORDS = {
    "eye": {"eyem"},
    "chew": {"chew"},
    "muscle": {"musc", "shiv"},
    "non_physiological": {"elec", "elpp"},
}

```

```

# Seizure exceptions due to recognition in the TUEV corpus
SEIZURE_TOKENS = {"tcsz", "cpsz", "gnsz", "fnsz"}

# Neutral label
BACKGROUND_LABEL = "bckg"

# Process to analyze data
def split_compound_label(label):
    return set(label.lower().split("_"))

```

Window generation

```

[ ]: # Temporal (windowing) interaction by labels
def label_windows(annotations_df, window_size_sec):
    rows = []
    group_cols = ["Patient", "Session", "Section", "Montage"]

    for keys, session_group in annotations_df.groupby(group_cols):
        patient, session, section, montage = keys
        max_time = session_group["stop_time"].max()
        n_windows = int(max_time // window_size_sec)

        for w in range(n_windows):
            win_start = w * window_size_sec
            win_end = win_start + window_size_sec

            overlapping = session_group[
                (session_group["start_time"] < win_end) &
                (session_group["stop_time"] > win_start)
            ]

            raw_labels = list(overlapping["label"].unique())
            channels_involved = overlapping["channel"].nunique()

            all_tokens = set()
            for lbl in raw_labels:
                all_tokens |= split_compound_label(lbl)

            row = {
                "Patient": patient, "Session": session,
                "Section": section, "Montage": montage,
                "window_size": window_size_sec,
                "start": win_start, "end": win_end,
                "raw_labels": raw_labels,
                "n_channels_annotated": channels_involved,
            }

            for group_name, keywords in ARTIFACT_KEYWORDS.items():

```



```

        row[group_name] = int(bool(all_tokens & keywords))

        genuine_cooccurrence = False
        for lbl in raw_labels:
            tokens = split_compound_label(lbl)
            categories_hit = {g for g, kws in ARTIFACT_KEYWORDS.items() if
↪tokens & kws}
            if len(categories_hit) > 1:
                genuine_cooccurrence = True
                break

        row["genuine_cooccurrence"] = int(genuine_cooccurrence)
        categories_present = sum(row[g] for g in ARTIFACT_KEYWORDS)
        row["weak_overlap"] = int(categories_present > 1 and not
↪genuine_cooccurrence)

        row["has_seizure_activity"] = int(bool(all_tokens & SEIZURE_TOKENS))

        row["is_unreviewed"] = int(len(all_tokens) == 0)
        row["is_background"] = int(
            BACKGROUND_LABEL in all_tokens
            and not any(row[g] for g in ARTIFACT_KEYWORDS)
            and not row["has_seizure_activity"]
        )

        rows.append(row)

    return pd.DataFrame(rows)

t0 = time.time()
segments_df = label_windows(data_sessions_artifact_df, window_size_sec=1)
print(f"{time.time()-t0:.2f}s para {len(segments_df)} ventanas totales")

```

256.22s para 344482 ventanas totales

```

[55]: # Visualization
display(segments_df)

```

	Patient	Session	Section	Montage	window_size	start	end	
↪	raw_labels	n_channels	annotated	eye	chew	muscle	non_physiological	↪
↪	genuine_cooccurrence	weak_overlap	has_seizure_activity	is_unreviewed				↪
↪	is_background							
0	aaaaaaju	s005	t000	01_tcp_ar	1	0	1	↪
↪	[]		0	0	0	0		↪
↪	0		0		0	1	0	
1	aaaaaaju	s005	t000	01_tcp_ar	1	1	2	↪
↪	[]		0	0	0	0		↪
↪	0		0		0	1	0	

2	aaaaaaaju	s005	t000	01_tcp_ar	1	2	3	
↪ []		0	0	0 0		0		
↪ 0		0		0	1		0	
3	aaaaaaaju	s005	t000	01_tcp_ar	1	3	4	
↪ []		0	0	0 0		0		
↪ 0		0		0	1		0	
4	aaaaaaaju	s005	t000	01_tcp_ar	1	4	5	
↪ []		0	0	0 0		0		
↪ 0		0		0	1		0	
...	...	...	...	...	...	...	...	
↪ ...	...	...	...	...	...	...	...	
↪ ...	...	...	...	...	...	...	...	
344477	aaaaatjz	s001	t000	03_tcp_ar_a	1	1893	1894	
↪ [bckg]		22	0 0 0			0		
↪ 0		0		0		0	1	
344478	aaaaatjz	s001	t000	03_tcp_ar_a	1	1894	1895	
↪ [bckg]		22	0 0 0			0		
↪ 0		0		0		0	1	
344479	aaaaatjz	s001	t000	03_tcp_ar_a	1	1895	1896	
↪ [bckg]		22	0 0 0			0		
↪ 0		0		0		0	1	
344480	aaaaatjz	s001	t000	03_tcp_ar_a	1	1896	1897	
↪ [bckg]		22	0 0 0			0		
↪ 0		0		0		0	1	
344481	aaaaatjz	s001	t000	03_tcp_ar_a	1	1897	1898	
↪ [bckg]		22	0 0 0			0		
↪ 0		0		0		0	1	

[344482 rows x 18 columns]

```
[58]: # Percentage of data that is unreviewed
print("\n>> Percentage of unreviewed data according to windowing analysis")
print(segments_df["is_unreviewed"].mean()*100, "%") # si esto es ~1.0, algo
↪ está mal
```

```
>> Percentage of unreviewed data according to windowing analysis
48.7116885457005 %
```

```
[57]: ## Results analysis for windowing
print("\n>> Analysis of annotations according to windows of 1s to all corpus")
display(segments_df.head(30))

# Manual visualization
sample = data_sessions_artifact_df[
    (data_sessions_artifact_df["Patient"]=="aaaaaaaju") &
    (data_sessions_artifact_df["Session"]=="s005")]
```

```

]
print("\n>> Manual correlation to verify data")
print(sample[["channel","start_time","stop_time","label"]].head(5))

```

>> Analysis of annotations according to windows of 1s to all corpus

	Patient	Session	Section	Montage	window_size	start	end	raw_labels	
0	aaaaaaaju	s005	t000	01_tcp_ar	1	0	1	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
1	aaaaaaaju	s005	t000	01_tcp_ar	1	1	2	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
2	aaaaaaaju	s005	t000	01_tcp_ar	1	2	3	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
3	aaaaaaaju	s005	t000	01_tcp_ar	1	3	4	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
4	aaaaaaaju	s005	t000	01_tcp_ar	1	4	5	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
5	aaaaaaaju	s005	t000	01_tcp_ar	1	5	6	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
6	aaaaaaaju	s005	t000	01_tcp_ar	1	6	7	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
7	aaaaaaaju	s005	t000	01_tcp_ar	1	7	8	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
8	aaaaaaaju	s005	t000	01_tcp_ar	1	8	9	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
9	aaaaaaaju	s005	t000	01_tcp_ar	1	9	10	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
10	aaaaaaaju	s005	t000	01_tcp_ar	1	10	11	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		
11	aaaaaaaju	s005	t000	01_tcp_ar	1	11	12	[]	␣
↪		0	0	0	0	0		0	␣
↪	0			0	1		0		

12	aaaaaaaju	s005	t000	01_tcp_ar	1	12	13	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
13	aaaaaaaju	s005	t000	01_tcp_ar	1	13	14	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
14	aaaaaaaju	s005	t000	01_tcp_ar	1	14	15	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
15	aaaaaaaju	s005	t000	01_tcp_ar	1	15	16	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
16	aaaaaaaju	s005	t000	01_tcp_ar	1	16	17	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
17	aaaaaaaju	s005	t000	01_tcp_ar	1	17	18	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
18	aaaaaaaju	s005	t000	01_tcp_ar	1	18	19	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
19	aaaaaaaju	s005	t000	01_tcp_ar	1	19	20	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
20	aaaaaaaju	s005	t000	01_tcp_ar	1	20	21	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
21	aaaaaaaju	s005	t000	01_tcp_ar	1	21	22	[]	0	0
↪		0	0	0		0				0
↪	0			0	1		0			0
22	aaaaaaaju	s005	t000	01_tcp_ar	1	22	23	[eyem]	0	0
↪		8	1	0		0				0
↪	0			0	0		0			0
23	aaaaaaaju	s005	t000	01_tcp_ar	1	23	24	[eyem]	0	0
↪		8	1	0		0				0
↪	0			0	0		0			0
24	aaaaaaaju	s005	t000	01_tcp_ar	1	24	25	[eyem]	0	0
↪		8	1	0		0				0
↪	0			0	0		0			0
25	aaaaaaaju	s005	t000	01_tcp_ar	1	25	26	[eyem]	0	0
↪		8	1	0		0				0
↪	0			0	0		0			0
26	aaaaaaaju	s005	t000	01_tcp_ar	1	26	27	[eyem]	0	0
↪		8	1	0		0				0
↪	0			0	0		0			0

```

27 aaaaaaju s005 t000 01_tcp_ar 1 27 28 [eyem]
↪      8 1 0 0 0 0 0
↪      0 0 0
28 aaaaaaju s005 t000 01_tcp_ar 1 28 29 [eyem]
↪      8 1 0 0 0 0 0
↪      0 0 0
29 aaaaaaju s005 t000 01_tcp_ar 1 29 30 [eyem]
↪      8 1 0 0 0 0 0
↪      0 0 0

```

>> Manual correlation to verify data

```

channel start_time stop_time label
150667 FP1-F7 22.9737 30.0688 eyem
150668 FP1-F7 136.7987 140.1117 eyem
150669 FP1-F7 145.0133 148.0498 eyem
150670 F7-T3 22.9737 30.0688 eyem
150671 F7-T3 136.7987 140.1117 eyem

```

```

[40]: data_sessions_artifact_df["label"]
data_sessions_artifact_df["label"].value_counts()
data_sessions_artifact_df.groupby("label")["WindowSize"].describe()

```

```

[40]: count mean std min 25% 50%
75% max
label
bckg 3122.0 238.982662 349.265782 1.0000 25.76230 91.85680
376.74490 2352.0111
chew 6482.0 10.527261 21.001009 0.2945 1.81320 4.79180
11.12725 233.0350
chew_elec 152.0 128.375164 155.699448 1.9427 7.25250 27.36110
272.63190 434.2833
chew_musc 243.0 29.209626 24.916542 1.0986 9.97180 16.59290
48.25780 79.4469
cpsz 484.0 104.582705 52.769340 13.3657 63.63180 112.58865
137.67480 238.4000
elec 33130.0 6.518679 49.752302 0.0222 0.30830 0.91670
3.84120 1858.6248
elpp 172.0 1.188781 0.350740 0.9663 0.99580 1.00180
1.51150 1.9858
eyem 38569.0 5.276276 21.436560 0.1389 0.99910 1.83160
4.13880 1356.1207
eyem_chew 864.0 20.393904 91.740231 0.3828 0.97250 2.46070
5.90390 584.8844
eyem_elec 2422.0 9.222971 18.758838 0.1838 1.29220 3.45580
8.05570 139.0160
eyem_musc 18677.0 6.281791 36.248278 0.0479 0.98090 2.09840
4.90430 1581.1441

```

eyem_shiv	45.0	16.919778	13.363381	3.4966	4.01910	4.01910
30.43060	30.4306					
fnsz	1246.0	57.949342	51.139852	3.8429	26.06940	40.10760
77.41400	255.6544					
gnsz	1223.0	55.667833	50.969928	3.8292	17.98000	34.97330
91.03780	209.0367					
musc	51052.0	11.252728	64.869857	0.0889	1.25810	2.95900
6.43880	2402.1625					
musc_elec	7651.0	25.524313	123.857603	0.1223	2.23470	5.25440
13.52690	1688.2039					
shiv	613.0	6.938291	7.088396	0.5333	3.82220	4.48660
7.13170	33.1808					
shiv_elec	1.0	3.822200	NaN	3.8222	3.82220	3.82220
3.82220	3.8222					
tcsz	147.0	126.283052	43.682923	12.8930	98.44285	151.10640
157.08590	166.1134					

```
[ ]: # Dataset balance by new labels (not 19)
artifact_counts = segments_df[list(ARTIFACT_KEYWORDS.keys())].sum()

artifact_percent = (
    artifact_counts / artifact_counts.sum() * 100
).round(2)

display(pd.DataFrame({
    "Windows": artifact_counts,
    "Percentage": artifact_percent
})))

# Patients per label
print("\n>> Patients and label distribution")
patients_per_class = {
    artifact:
        segments_df.loc[
            segments_df[artifact] == 1,
            "Patient"
        ].nunique()
    for artifact in ARTIFACT_KEYWORDS
}
display(pd.Series(patients_per_class))

# Windows per patient
print("\n>> Windows per patient distribution")
windows_per_patient = (
    segments_df.groupby("Patient").size()
)
display(windows_per_patient.describe())
```

```

windows_per_patient.plot.hist()

# Duration per label
duration = artifact_counts / 3600
display(duration.rename("Hours"))

# Original annotations duration
data_sessions_artifact_df.groupby("label")["WindowSize"].describe()

```

	Windows	Percentage
eye	70791	32.79
chew	5699	2.64
muscle	86154	39.90
non_physiological	53274	24.67

>> Patients and label distribution

eye	196
chew	30
muscle	205
non_physiological	196

dtype: int64

>> Windows per patient distribution

count	213.000000
mean	1617.286385
std	1239.310067
min	11.000000
25%	1165.000000
50%	1238.000000
75%	1498.000000
max	10098.000000

dtype: float64

eye	19.664167
chew	1.583056
muscle	23.931667
non_physiological	14.798333

Name: Hours, dtype: float64

	count	mean	std	min	25%	50%
75%	max					
label						
bckg	3122.0	238.982662	349.265782	1.0000	25.76230	91.85680
	376.74490	2352.0111				
chew	6482.0	10.527261	21.001009	0.2945	1.81320	4.79180
	11.12725	233.0350				

chew_elec	152.0	128.375164	155.699448	1.9427	7.25250	27.36110
272.63190	434.2833					
chew_musc	243.0	29.209626	24.916542	1.0986	9.97180	16.59290
48.25780	79.4469					
cpsz	484.0	104.582705	52.769340	13.3657	63.63180	112.58865
137.67480	238.4000					
elec	33130.0	6.518679	49.752302	0.0222	0.30830	0.91670
3.84120	1858.6248					
elpp	172.0	1.188781	0.350740	0.9663	0.99580	1.00180
1.51150	1.9858					
eyem	38569.0	5.276276	21.436560	0.1389	0.99910	1.83160
4.13880	1356.1207					
eyem_chew	864.0	20.393904	91.740231	0.3828	0.97250	2.46070
5.90390	584.8844					
eyem_elec	2422.0	9.222971	18.758838	0.1838	1.29220	3.45580
8.05570	139.0160					
eyem_musc	18677.0	6.281791	36.248278	0.0479	0.98090	2.09840
4.90430	1581.1441					
eyem_shiv	45.0	16.919778	13.363381	3.4966	4.01910	4.01910
30.43060	30.4306					
fnsz	1246.0	57.949342	51.139852	3.8429	26.06940	40.10760
77.41400	255.6544					
gnsz	1223.0	55.667833	50.969928	3.8292	17.98000	34.97330
91.03780	209.0367					
musc	51052.0	11.252728	64.869857	0.0889	1.25810	2.95900
6.43880	2402.1625					
musc_elec	7651.0	25.524313	123.857603	0.1223	2.23470	5.25440
13.52690	1688.2039					
shiv	613.0	6.938291	7.088396	0.5333	3.82220	4.48660
7.13170	33.1808					
shiv_elec	1.0	3.822200	NaN	3.8222	3.82220	3.82220
3.82220	3.8222					
tcsz	147.0	126.283052	43.682923	12.8930	98.44285	151.10640
157.08590	166.1134					

